CS 4120 Overview Document
Programming Assignment 6: Optimization
Chenwei Hou (ch2352), Thomas Lu (cl2625), Steven Yu (ssy38)

Fully qualified class name of your main program: since we are using C++, our main function must be in the global namespace, so this does not apply.

See README for instructions on building and running the program.

Summary

We have implemented an optimizing compiler for the Eta Programming Language. The compiler is capable of performing all required optimizations, including dead code elimination, copy propagation, and register allocation. We implemented dead code elimination and copy propagation together as a cascaded analysis and register allocation using an Appel-style graph coloring algorithm.The compiler additionally performs local value numbering and flow-sensitive conditional constant propagation. Finally, the compiler supports the Rho Language Extension while maintaining backward compatibility to Eta.

Specification

As with previous assignments, we designed formal specifications for implementing the optimizations. The exception is register allocation, since a comprehensive documentation for the graph coloring algorithm could be found in Appel's textbook and the live variable analysis is also standard modulo ABI and architectural considerations. We have also updated previous specs for parsing, type checking, and code generation to support the Rho Language Extension. These specifications continue to act as the authoritative source for the compiler's behavior. Nevertheless, we highlight certain crucial ambiguities that we needed to resolve when designing the specification:

1. When performing local value numbering, we only checked for equivalent expressions that could be detected by applying commutative laws at the top level of the AST (e.g. a + b and b + a). We did not perform other forms of equality saturation.
2. When deciding which node to spill in the node for register allocation we used the heuristic total_use + total_def / degree where degree is the degree of node in interference graph.
3. The majority of the ambiguities of the spec lie at the module system for the Rho language extension. Importantly, we decided to forbid cyclic use chains because module systems similar to those for Rho (i.e. the OCaml module system) does such. We resolve a more crucial ambiguity involving the semantics of use, such that the implementation file is only required to implement the functions and records explicitly declared in the interface file. The rationale is that we interpret uses in programs and interfaces as exposing the names and signatures of function and records from the other interfaces for the compiler (and the linker) but not "copying" the actual code from those interfaces. The problem with otherwise is that it breaks modularity: if a module (say, zoo) needs a record

definition (say, animal) of another module in its method signatures, then the module also needs to reimplement everything in that module, which would be problematic. Another detail worth highlighting is that the compiler will parse and typecheck every .ri interface files in the supplied libpath. The reason is that we wanted to keep parsing and typechecking stages strictly hierarchical and not have the typechecker call the parser on-the-fly upon every use statement in an interface. Finally, we needed to take some liberties in designing the Rho type system. As required, null can be assigned to every record and array type but not int or bool types. The reason for this is that the value null represents a pointer, but int and bool values are not pointers. Additionally, we say that accessing the field values of a null value has the same type of accessing an empty array. This is because their type-theoretic semantics is equivalent in that they could result in the production of no run-time values.

4. We took extra care to ensure backwards compatibility from Rho into Eta. Specifically, the compiler selects the appropriate parsing rule to use depending on the extension of the supplied file. For example, if a program has an .eta extension, then new keywords in Rho such as null, record, and break can be used as legal identifiers for variables. Nevertheless, the one exception is that we use a universal stringifier for both Eta and Rho ASTs, so now a stringfied Eta interface AST will contain another pair of parentheses on the toplevel representing an empty set of use statements.

Design and Implementation

The implementation for this programming assignment is naturally divided into the following main parts: implementing the Rho language extension and one part for each of the optimizations. We now discuss each part in more detail:

1. Copy propagation and dead code elimination: for these optimizations, we performed the textbook dataflow analyses with a worklist algorithm, and ran the two optimizations in alternation until convergence.
2. Conditional Constant Propagation: this is again the textbook dataflow analysis with worklist algorithm, employing the semantically appropriate abstract interpretations.
3. Local Value Numbering: although a local flow-insensitive optimization, we needed to explicitly maintain different value number mappings for different types of expressions, which makes the implementation harder to implement than anticipated.
4. Register allocation: this is the most involved optimization. Although we have a comprehensive documentation of a graph-coloring-based register allocation algorithm in Appel's textbook, the implementation of which still contained many pitfalls. The first is the definition of the use and def sets in liveness analysis for different abstract assembly instructions. One especially tricky case to notice is that if the function returns more than two values, then the function uses an extra argument to contain a pointer to the extraneous return values, which must be reflected in the use sets for the calls to that function. Another complication arises when translating instructions that uses or defines two spilled temporaries that are coalesced together but only writes to one of them, where the translated instruction must only save to the rewritten temporary to prevent the subsequent save from overwriting the first.

5. The Rho Language Extension: although not hard to implement given the formal specification, the extension exposed the rigidity of our codebase and many aspects of our compiler frontend have to be restructured to accommodate the extension. For example, the AST nodes for programs and interfaces are now of a single class due to many overlaps in the variety of declarations available to programs and interfaces.

Testing

Due to the implementation being divided into mutually exclusive parts, it is natural that the test plan is also divided among these parts. For each optimization, we reran existing test cases for PA4 and PA5 and also devised additional test cases, some of which is to be submitted as our benchmark test cases, specifically aimed to test the effectiveness and robustness of the specific optimization. For these test cases, we not only test for correctness, but also the improvement in runtime after the specific optimization is applied. Additional care has been put into tests for register allocation as the initial unit test suite fails to sufficiently test the various complexities of the register allocation algorithm, such as spilling and coalescing. We also designed unit test suites for the Rho language extension and ensured that it not only tests for code generation but also all other components of the compilation pipeline, with special emphasis on the edge cases of type checking such as overlapping use chains. After all parts are finished, we perform all test cases on the completed compiler as integration testing.

Work Plan

We developed the various optimizations in parallel. In particular, Thomas Lu implemented the Rho language extension and adopted Appel's register allocation algorithm, Steven Yu implemented the liveness analyzer and code generator at abstract assembly level, while Chenwei Hou implemented the remaining optimizations at the IR level. Steven Yu also contributed to refining the test plan and packaging the code for shipping. All of us contributed roughly equally to the project.

AI Usage

We have used AI to assist various parts of our system, and, in rare cases, to generate certain pieces of well-scoped code. Regardless of the use case, we meticulously inspected each line of AI output for correctness.

Known problems

We foresee no problems with our implementation.

As mentioned in previous overview documents, we do not support escaping double quotes, which leads to the test case failures involving pointexample.rh. If that Rho file is edited to use unicode escapes (i.e. \" → \x{22}), the test case will pass as expected. For your convenience,

we have edited tests/provided/pa6/pointexample.rh to use \x{22} and all tests pass as expected. We do not think this is an issue, because we have documented it before.

Comments

We survived.